

Pytorch - How to normalize/transform data manually for DataLoader

Asked 3 years, 5 months ago Modified 2 years, 8 months ago Viewed 9k times



Vote



I am following along with a LinkedInLearning tutorial for neural networks. I am trying to follow along using a different dataset than in the tutorial, but applying the same techniques to my own dataset. I am struggling with figuring out how to normalize/transform my data in the same way they do, because they are using some built in functionality that I do not know how to reproduce.

Here is an example of what they are doing:

```
from torchvision import datasets, transforms

mean, std = (0.5,), (0.5,)

# Create a transform and normalise data
transform = transforms.Compose([transforms.ToTensor(),
                               transforms.Normalize(mean, std)
                              ])

# Download FMNIST training dataset and load training data
trainset = datasets.FashionMNIST('~/.pytorch/FMNIST/', download=True, train=True,
transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
```

they are creating a transform, and then just passing it straight into this `FashionMNIST` method, which seems to be doing some sort of automatic transforming for the `trainset`.

I want to do a similar thing, but for my dataset, there is no built in `FashionMNIST` method. How would I replicate it?

Here's what I'm doing/know how to do:

```
import pandas as pd

df = pd.read_csv('../input/sign-language-mnist/sign_mnist_train.csv')
trainloader = torch.utils.data.DataLoader(df, batch_size = 64, shuffle = True)
```

How would I go about applying the same `transform` to my `df` without the help of this built in `FashionMNIST` method?

[python](#) [pandas](#) [pytorch](#) [normalization](#) [Edit tags](#)

[Share](#) [Edit](#) [Follow](#) [Close](#) [Flag](#)

asked Sep 23, 2021 at 0:47



kdeep

394 ● 8 ● 18



You need to build a custom Pytorch dataset to put into your dataloader

2



```
class MNistDataset:
    def __init__(self, df):
        self.df = self.custom_norm_function(df)
        self.device = T.device('cuda:0' if T.cuda.is_available() else 'cpu')

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        image = self.df.loc[idx, 'image']
        label = self.df.loc[idx, 'label']
        return image, label

    def custom_norm_function(self, df):
        df = normalize(df)
        return df
```

Where you define your "custom_norm_function" as needed. Then put it in your dataloader.

```
dataset = MNistDataset(df)
trainloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

You can read more here -> https://pytorch.org/tutorials/beginner/data_loading_tutorial.html

Share Edit Follow Flag

answered May 28, 2022 at 3:54



Stupid Loser

96 ● 1 ● 5



-1



This post is hidden. It was [deleted](#) 3 years ago by the post author.

The `transforms.Compose` split the work into two functions: `transform.toTensor` and `transforms.Normalize`.

1) transforms.toTensor

From the docs:

Convert a `PIL.Image` or `numpy.ndarray` to tensor.

Converts a `PIL.Image` or `numpy.ndarray` ($H \times W \times C$) in the range $[0, 255]$ to a `torch.FloatTensor` of shape $(C \times H \times W)$ in the range $[0.0, 1.0]$.

<https://pytorch.org/docs/0.2.0/modules/torchvision/transforms.html#ToTensor>

Basically, this function will transform your inputs $[0, 255]$ into values of $[0, 1]$ for each channel in your image dataset.

2) transforms.Normalize

This function subtracts your data by the mean and divides by the standard deviation, as presented in the docs:

```
torchvision.transforms.Normalize(mean, std, inplace=False)
```

Normalize a tensor image with mean and standard deviation. This transform does not support PIL Image. Given mean: (mean[1],...,mean[n]) and std: (std[1],...,std[n]) for n channels, this transform will normalize each channel of the input torch. Tensor i.e., $\text{output[channel]} = (\text{input[channel]} - \text{mean[channel]}) / \text{std[channel]}$

<https://pytorch.org/vision/stable/transforms.html>

So, to have the same effect you should simply do the same with your data. First transform from [0, 255] to [0, 1], and after that using the Z-score equation: $z = (x - \text{mean}) / \text{std}$, where X is your data, Normalize your data.

Share Edit Follow **Undelete** Flag

edited Sep 23, 2021 at 2:40



Dharman ♦

33.4k ● 27 ● 101 ● 147

answered Sep 23, 2021 at 2:35



Romero_91

425 ● 4 ● 15

Comments disabled on deleted / locked posts / reviews